



Generics

Modulübersicht

- Einsatz und Verwendung von Generics
- generische Klassen
- generische Methoden
- Constraints
- Vererbung von generischen Klassen
- Kovarianz, Kontravarianz, Invarianz

Generics

- Platzhalter für Datentypen in Klassen und Methoden

```
void Method<T>(T obj)      Method<object>(new object());  
{                          Method(new object());  
    ...                    Method("Hello World");  
}
```

- Beispiele im .NET Framework:
List<T>, Stack<T>, Dictionary<TKey, TValue>, Comparer<T>
Action<T>, Func<T>, EventHandler<T> ...

Eigene Klasse/Methode mit Generic

- Ein oder mehrere Generische Typargumente (<T>) bei Klassen-/Methodendefinition hinzufügen
- T kann statt Typen positioniert werden
 - Variable
 - Parameter
 - Rückgabewert
 - Vererbung
 - Andere Generics
 - ...

Constraints

- Einschränkungen für die Wahl des Datentypen
- Arten von Constraints:

```
class InterfaceConstraint<T>   where T : IDisposable { }  
class KlassenConstraint<T>    where T : Beispielklasse { }  
class WertetypConstraint<T>   where T : struct { }  
class ReferenztypConstraint<T> where T : class { }  
class KonstruktorConstraint<T> where T : new() { }
```

Constraints können auch kombiniert werden. Beispiel: 2 Interfaces und eine Klasse.

Vererbung

- Ist die Basisklasse generisch, kann die Subklasse den Typenparameter übernehmen
- Soll die Subklasse nicht generisch sein, kann man einen konkreten Typen einsetzen

```
class Basisklasse<T> { }  
class SubklasseOffen<T> : Basisklasse<T> { }  
class SubklasseGeschlossen: Basisklasse<int> { }  
  
class BasisklasseMitEinschränkungen<T> where T : class { }  
class SubklasseOffenMitEinschränkungen<T> : Basisklasse<T> where T : class { }  
class SubklasseGeschlossenMitEinschränkungen : Basisklasse<string> { }
```

Kovarianz (out)

- Eine Instanz eines kovarianten generischen Typen ist implizit zu einem generischen Typen konvertierbar, der einen weniger spezifischeren generischen Typenparameter besitzt

```
[TypeDependencyAttribute("System.SZArrayHelper")]  
public interface IEnumerable<out T> : IEnumerable  
  
IEnumerable<Tier> tiere = Enumerable.Empty<Tier>();  
IEnumerable<Hund> hunde = Enumerable.Empty<Hund>();  
  
tiere = hunde;  
// hunde = tiere; // Typenfehler
```

Dies funktioniert aber NUR, weil IEnumerable Elemente des Typs T erzeugt, jedoch nicht konsumiert.

Ein Objekt, das Hunde produziert, kann genauso genutzt werden, wie ein Objekt, das Tiere produziert. (Hund ist laut Polymorphie ein Tier, somit OK)

-> out - Keyword

Kontravarianz (in)

- Eine Instanz eines kontravarianten generischen Typen ist implizit zu einem generischen Typen konvertierbar, der einen spezifischeren generischen Typenparameter besitzt

```
public interface IComparer<in T>

    IComparer<Tier> tierVergleicher = null;
    IComparer<Hund> hundVergleicher = null;

    hundVergleicher = tierVergleicher;
    // tierVergleicher = hundVergleicher; // Typenfehler
```

Dies funktioniert nur, weil IComparer ein Objekt des Typs T konsumiert (entgegennimmt), aber nicht produziert (zurückgibt).
Ein Objekt, das zwei Tiere vergleichen kann, kann auch zwei Hunde vergleichen.
(KEINE RÜCKGABE VON HUND SELBST !!!)

-> in - Keyword

Invarianz

- Es besteht keine Beziehung zwischen invarianten Typen, da man sowohl Werte konsumieren als auch produzieren kann

```
[DefaultMember("Item")]
[TypeDependencyAttribute("System.SZArrayHelper")]
public interface IList<T> : ICollection<T>, IEnumerable<T>, IEnumerable

IList<Hund> hunde = new List<Hund>();
IList<Tier> tiere = new List<Tier>();
// tiere = hunde; // Typenfehler

// Wenn IList kovariant wäre, könnte man Elemente vom falschen Typ hinzufügen
IList<Tier> zoo = new List<Katze>(); // Wenn das erlaubt wäre...
zoo.Add(new Hund()); // ... wäre das auch erlaubt

// Wenn IList kontravariant wäre, könnte man Elemente vom falschen Typ zurückbekommen
IList<Hund> zwinger = new List<Tier>(); // Wenn das erlaubt wäre ...
Hund dog = zwinger[0]; // ... wäre das auch erlaubt
```

There is no subtype relationship for invariant Types, because you can put values in AND take values out

Kovarianz: $\text{IEnumerable<Dog>} \rightarrow \text{IEnumerable<Animal>}$
spezifisch allgemein

Kontravarianz: $\text{Action<Animal>} \rightarrow \text{Action<Dog>}$
allgemein spezifisch

Warum die Richtung bei Kontravarianz umgekehrt ist: Eine Funktion, die *jedes Animal* verarbeiten kann, kann natürlich auch dort eingesetzt werden, wo lediglich eine Funktion für Dog verlangt wird.